Ders 12 Genişletilmiş Notu: Uygulama: Otobüs Bileti Senaryosu İçin UML Sınıf, Sıralama, Etkinlik ve Durum Diyagramları

Ders 12 Genişletilmiş Notu: Uygulama: Otobüs Bileti Senaryosu İçin UML Sınıf, Sıralama, Etkinlik ve Durum Diyagramları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Uygulamanın Amacı ve Senaryo
- 2. UML Sınıf Diyagramı
- 3. UML Sıralama Diyagramı
- 4. Etkinlik Diyagramı
- 5. Koltuk İçin Durum Diyagramı
- 6. Diyagramlar Arası Tutarlılık ve Eksikler
- Örnek Sorular
- Dersin Toparlaması

1. Uygulamanın Amacı ve Senaryo

Bu ders bir uygulama dersidir. Prof. Dr. Oya Kalıpsız dersi açtı ve uygulamayı bir araştırma görevlisine bıraktı. Uygulamada tek bir senaryo üzerinden dört farklı UML diyagramı çizildi. Diyagramlar ekranda bir UML çizim programında (kısa nota göre Violet UML Editor) adım adım oluşturuldu. Bu not altyazıdan hazırlandığı için diyagramların görsel ayrıntıları, ancak anlatımda söylendiği ölçüde verilebiliyor. Ekrandaki sınıf ve durum diyagramının yazımı, kısa notu hazırlarken alınan kareden aktarılmıştır.

1.1 Senaryonun metni

Senaryo, internetten otobüs bileti almaya benzer. Asistanın deyişiyle “basitçe bir otobüs bilet sitesi” düşünülmelidir: Orada bir rezervasyon yapılır ve en sonunda bilet alınır.

Sistemden istenen şudur: Bir yolcunun yolculuk etmek istediği seferi, o seferdeki otobüsü ve o otobüsteki koltuğu seçmesini sağlayan ve bu bilgilerin rezervasyon bilgisi olarak kaydedildiği bir sistem tasarlanacaktır. Senaryo, bir kullanım durumu (use case) kalıbıyla verilmiştir:

- **Birinci aktör (primary actor):** Yolcu.
- **Diğer ilgililer (stakeholders):** Bilet satış sistemi.
- **İlgi alanı:** Yolcu, koltuğunu kolayca seçip biletini kolayca alabilmek ister.
- **Ön koşul (precondition):** Bilet satış sistemine tüm sefer bilgileri girilmiştir.
- **Son koşul (postcondition):** Yolcu biletini satın almıştır.

1.2 Ana senaryo

Ana akış (main success scenario) şu adımlardan oluşur:

1. Yolcu, yapmak istediği seyahatin bilgilerini girer.
2. Sistem bu bilgilere uyan otobüs seferlerini gösterir.
3. Yolcu bir otobüs seçer.
4. Yolcu seçtiği otobüste bir koltuk seçer. Sistem koltuğun uygunluğunu kontrol eder.
5. Yolcu seçimlerini yaptıktan sonra rezervasyon bilgilerini girer. Koltuk sistemde "rezerve" olarak işlenir.
6. Yolcu ödemeyi yapar. Koltuk için biletin alındığı sisteme işlenir.

Dördüncü adımdaki uygunluk kontrolü yalnız koltuğun boş ya da dolu olmasıyla sınırlı değildir. Asistan, belirli kurallar olabileceğini, bir koltuk boş olsa bile o yolcunun onu alamayabileceğini söyledi. Yolcu buna göre koltuk seçer.

1.3 Alternatif senaryolar

İki alternatif akış vardır:

- **Alternatif 1, uygun olmayan koltuk:** Yolcu uygun olmayan bir koltuğu, örneğin dolu bir koltuğu seçmeye çalışır. Sistem buna izin vermez ve yolcudan yeni bir koltuk seçmesini ister.
- **Alternatif 2, ödeme süresi:** Yolcu, seferin başlangıcına 12 saat kalana kadar rezervasyon için ödeme yapıp koltuğunu biletlemezse sistem bu rezervasyonu iptal eder ve koltuğu yeniden boş olarak işaretler.

Asistan ikinci alternatifi anlatırken bir yerde bunu "rezervasyondan sonra yolcuya ödeme için 12 saat süre veriliyor" diye ifade etti. Durum diyagramını çizerken bu ifadeyi kendisi düzeltti: Kural "12 saat içinde" değil, "sefer başlangıcına 12 saat kalana kadar" ödemedir (bkz. Bölüm 5).

1.4 İstenen dört diyagram

Senaryo için dört diyagram istendi:

1. UML sınıf şeması (class diagram).
2. Sınıf şemasındaki ilişki ve metotlara uygun bir UML sıralama diyagramı (sequence diagram).
3. Bir etkinlik diyagramı (activity diagram).
4. Bir koltuğun içinde bulunabileceği durumları ve bunlar arasındaki olası geçişleri ayrıntılandıran bir UML durum diyagramı (state diagram).

2. UML Sınıf Diyagramı

Sınıf şemasında, yazılımın içindeki sınıflar ve bunların arasındaki ilişkiler tanımlanır. Asistan diyagramı iki aşamada kurdu: Önce sınıf kutularını yerleştirip ilişkileri çizdi, sonra her sınıfın niteliklerini (attributes) ve metotlarını tek tek doldurdu.

2.1 Sınıfların bulunması

Asistan önce üç temel sınıfla başladı:

- **Yolcu:** Senaryonun birinci aktörü.
- **RezervasyonEkranı:** Yolcu rezervasyon yapacağı için, yolcunun etkileşimde bulunduğu ekran.
- **Sefer:** Rezervasyon bir sefere ve bir otobüse yapılacağı için sefer bilgileri gerekir.

Ardından bu üç sınıfın altına başka sınıflar ekledi:

- **Rezervasyon:** Rezervasyon ekranının altında, rezervasyon bilgilerini tutan sınıf.
- **Ödeme:** Açıklamada yolcunun rezervasyon yaptığı ama süresinde ödeme yapmazsa rezervasyonun iptal edildiği söyleniyor. Ödemeyi tutmak ve bu kuralı izlemek için ayrı bir sınıf açmak gerekir.
- **Otobüs:** Seferin altında, otobüs bilgilerini tutan sınıf.
- **Koltuk:** Otobüsün içinde, yolcunun seçeceği ve rezervasyonun yapılacağı koltuk bilgileri.

Böylece toplam 7 sınıf elde edildi.

2.2 Sınıflar arasındaki ilişkiler

Asistan sınıfları şöyle bağladı:

- Yolcu ile RezervasyonEkranı arasında bir ilişki (association).
- RezervasyonEkranı ile Rezervasyon arasında, Rezervasyon ile Ödeme arasında birer ilişki.
- RezervasyonEkranı ile Sefer arasında, Sefer ile Otobüs arasında birer ilişki.
- Otobüs ile Koltuk arasında bileşim (composition).

Otobüs ile koltuk için normal bir ilişki yerine bileşim seçilmesinin gerekçesi, ikisi arasında "kuvvetli bir ilişki" olmasıdır. Koltuk, otobüsün bir parçasıdır.

> **Ek bilgi (derste anlatılmadı):** UML'de bileşim, içi dolu bir elmasla bütün tarafına çizilir ve parçanın yaşam süresinin bütüne bağlı olduğunu gösterir: Bütün yok olursa parçalar da yok olur. Daha gevşek parça-bütün ilişkisi olan birleşme (aggregation) ise içi boş elmasla çizilir ve parça bütünden bağımsız var olabilir. Bu örnekte elmas Otobüs tarafındadır.

2.3 Görünürlük: private ve public

Asistan, nitelikleri private (-), onlara erişen metotları public (+) yaptı. Bunu önceki uygulamayla bağladı: Önceki uygulamada private gösterilmiş, public gösterilmemişti. Private bir üyeye yalnız sınıfın kendisi erişebilir. Public bir üyeye diğer sınıflar da erişebilir. Bu yüzden niteliklerin değerleri diğer sınıflara `get` ile başlayan public metotlar üzerinden açılır.

2.4 Sınıfların nitelik ve metotları

Aşağıdaki yazım, ekrandaki diyagramın kısa notta aktarılan halidir. Altyazıda anlatılan gerekçeler her sınıfın ardından verilmiştir.

```
Yolcu
  - ad: String
  - soyad: String
  - tcNo: String
  + getAd(): String
  + getSoyad(): String
  + getTcNo(): String

RezervasyonEkranı
  - seferler: Sefer()
  + seferAra(gidis: String, varis: String, gun: Date): boolean
  + gosterSeferBilgileri(sefer: Sefer): void
  + otobüsSec(busX: Otobüs): void
  + koltukSec(koltukY: Koltuk): void

Sefer
  - cıkısYeri: String
  - varısYeri: String
  - tarih: Date
  - otobüs: Otobüs
  + Sefer(cıkıs: String, varıs: String, tarih: Date, ...)
  + seferBilgileriniAl(): Sefer

Rezervasyon
  - rezervasyonNo: int
  - odeme: Odeme
  + rezervasyonYap(yolcu: Yolcu, sefer: Sefer, koltuk: Koltuk): void
  + odemeYap(rezervasyonNo: int): void

Ödeme
  - odemeTarihi: Date
```

```
  - odemeTutarı: double
  odemeAl(): void

Otobüs
  - plaka: String
  - koltuklar: Koltuk()
  + getPlaka(): String

Koltuk
  - koltukNo: int
  - koltukTipi: String
  - boslukDurumu: boolean
```

Yolcu. Yolcunun adı, soyadı ve TC numarası istenir. Üçü de `String` tutulur. Bunlar private nitelik olarak yazıldı. Diğer sınıfların bunlara ulaşabilmesi için aynı bilgiler `get` metotlarıyla public olarak eklendi; her biri `String` döndürür.

RezervasyonEkranı. Ekranın sefer bilgilerini tutması gerektiği için `seferler` adlı bir nitelik açıldı. Metotlar yolcunun yapacağı işlerden çıkarıldı:

- Yolcu önce uygun seferi bulmak için sefer arar. `seferAra` metodu gidiş yeri (`String`), varış yeri (`String`) ve seferin günü (`Date`) parametrelerini alır, `boolean` döndürür.
- Sefer bulunduktan sonra sefer bilgilerini gösteren bir metot eklendi (`gosterSeferBilgileri`).
- Yolcunun otobüs ve koltuk seçmesi gerektiği için `otobüsSec` ve `koltukSec` metotları eklendi. Asistan bunların "boş dönüleceğini", yani dönüş tipinin `void` olduğunu söyledi.

Sefer. Seferin çıkış yeri ve varış yeri (`String`) ile tarihi (`Date`) tutulur. Asistan bu bilgilere diğer sınıfların ulaşabileceği public metotlar da ekledi. Ayrıca, sefer bilgilerinin sonradan kullanılabilmesi için `seferBilgileriniAl` adlı public bir metot açtı.

Rezervasyon. Basitçe bir rezervasyon numarası ve ödeme bilgisi tutulur. Bunlar private olduğu için önlerine eksi işareti konuldu. Public `rezervasyonYap` metodu, rezervasyon yapabilmek için gereken üç bilgiyi alır: yolcu, sefer ve koltuk. Ödeme yapmak için ise rezervasyon numarası gerekir; bu yüzden `odemeYap` metodu rezervasyon numarasını parametre alır.

Ödeme. Ödeme tarihi (`Date`) ve ödeme tutarı private tutulur. Asistan tutar için "buradan sayı alacağız" dedi; kısa nottaki kareye göre tip `double`. Public olarak bir `odemeAl` metodu eklendi.

Otobüs. Otobüsün plakası ve koltukları nitelik olarak tutulur. Public olarak plakayı veren bir `get` metodu eklendi.

Koltuk. Koltuk numarası, koltuk tipi ve boşluk durumu tutulur. Koltuk tipi, tekli ya da çiftli gibi değerler aldığı için `String` seçildi. Boşluk durumu, senaryoda rezervasyondan önce koltuğun boş olup olmadığının kontrol edilmesi istendiği için eklendi. Asistan bu niteliklerin aynılarını `get` metotlarıyla public olarak da yazdığını söyledi.

2.5 Sınıf diyagramının özeti

Asistan diyagramı bitirince toparladı: Yolcu sınıfı var. Yolcu rezervasyon yapmak istediği için bir rezervasyon ekranı var. Rezervasyon bilgilerini tutan bir rezervasyon sınıfı ve ödeme için bir ödeme sınıfı var. Sefer bilgilerini tutan sefer sınıfı, otobüsün bilgilerini tutan otobüs sınıfı ve koltuğun bilgilerini tutan koltuk sınıfı var. Sistem temel olarak bu 7 sınıfla gerçekleştirildi.

3. UML Sıralama Diyagramı

3.1 Sıralama diyagramının yapısı

Sıralama diyagramı, belirli bir zaman sırasına göre düzenlenmiş nesnelerin etkileşimini gösterir. Elimizde belirli bir kullanım senaryosunda rol alan nesneler vardır. Diyagram bu nesneler arasında gidip gelen mesajları gösterir. Asistan diyagramı iki eksende anlattı:

- **Yatay eksen:** Nesneler üst tarafa yan yana yerleştirilir.
- **Dikey eksen:** Nesnelerin yaşamları boyunca hangi mesajları gönderip aldıkları, yukarıdan aşağı zaman sırasıyla gösterilir.

Her nesnenin altından inen çizgiye yaşam çizgisi (lifeline) denir. Bu çizgi, nesnenin sistem içindeki ömrünü ve o ömür boyunca ne yaptığını gösterir. Nesnenin etkin olduğu aralıklar yaşam çizgisi üzerine konan etkinlik çubuklarıyla (activation bar) gösterilir. Mesajlar, programdaki çağrı (call) ve oluşturma (create) mesajı araçlarıyla çizilir.

> **Hocanın vurgusu:** Sıralama diyagramı, az önce çizilen sınıf diyagramına uygun olarak çizilmelidir. Asistan her mesajı eklerken "tekrar class diagramımıza bakıyoruz" diyerek mesajları sınıf diyagramındaki metotlardan seçti.

3.2 Nesneler

Üst tarafa altı nesne yerleştirildi. Kısa nottaki adlandırmayla:

Sıra	Nesne	Sınıfı
1	`yolcu`	Yolcu
2	(adsız)	RezervasyonEkranı
3	(adsız)	Sefer
4	`busX`	Otobüs
5	`koltukY`	Koltuk
6	(adsız)	Rezervasyon

Ödeme sınıfı bu diyagrama nesne olarak konmadı.

3.3 Mesajlar adım adım

Asistan mesajları şu sırayla ekledi:

1. **Sefer arama.** Yolcu, rezervasyon ekranından önce sefer arar. Yolcudan rezervasyon ekranına `seferAra(parametreler)` mesajı çizildi; asistan parametreleri ayrıca yazmadı. Bu çağrının bir dönüş mesajı (return message) da vardır ve ekrandan yolcuya geri döner. Bu, sınıf diyagramında RezervasyonEkranı'ndaki `seferAra` metoduna karşılık gelir.
2. **Sefer bilgilerini alma.** Rezervasyon ekranı ile sefer arasında `seferBilgileriniAl()` mesajı çizildi. Dönüş mesajı olarak sefer, rezervasyon ekranına geri döndürülür.
3. **Sefer bilgilerini gösterme.** `gosterSeferBilgileri` rezervasyon ekranının kendi içinde yapması gereken bir iştir, başka bir sınıfa gitmez. Bu yüzden ekranın yaşam çizgisine ayrı bir etkinlik çubuğu eklendi ve mesaj ekranın kendisine dönen bir ok olarak çizildi (self message). Parametresi seferdir.
4. **Otobüs seçme.** Yolcudan rezervasyon ekranına `otobüsSec(busX)` mesajı.
5. **Koltuk seçme.** Yolcudan rezervasyon ekranına `koltukSec(koltukY)` mesajı.
6. **Boş koltukları listeleme.** Rezervasyon ekranından `busX` otobüsüne boş koltukları listeleyen bir mesaj gönderildi. Dönüşü koltuk bilgileridir.
7. **Boşluk durumunu sorma.** Otobüs ile koltuk arasında, koltuğun boşluk durumunu veren bir mesaj çizildi. Koltuk, dönüş mesajıyla boşluk bilgisini otobüse iletir.
8. **Rezervasyon yapma.** Rezervasyon ekranından rezervasyon nesnesine `rezervasyonYap` mesajı gönderildi. Parametreleri yolcu bilgisi, sefer bilgisi ve koltuk bilgisidir. Dönüş mesajı rezervasyon ekranına "rezervasyon onayı" olarak gelir.

Bu akış düz metin olarak şöyle özetlenebilir (`->` çağrı, `-->` dönüş mesajıdır):

```
yolcu            -> RezervasyonEkranı : seferAra(parametreler)
RezervasyonEkranı --> yolcu           : (dönüş)
RezervasyonEkranı -> Sefer            : seferBilgileriniAl()
Sefer            --> RezervasyonEkranı : sefer
RezervasyonEkranı -> RezervasyonEkranı : gosterSeferBilgileri(sefer)
yolcu            -> RezervasyonEkranı : otobüsSec(busX)
yolcu            -> RezervasyonEkranı : koltukSec(koltukY)
RezervasyonEkranı -> busX             : bosKoltuklarıListele()
busX             --> RezervasyonEkranı : koltuk bilgileri
busX             -> koltukY           : getBoslukDurumu()
koltukY          --> busX             : boşluk bilgisi
```

```
RezervasyonEkranı -> Rezervasyon       : rezervasyonYap(yolcu, sefer, koltukY)
Rezervasyon      --> RezervasyonEkranı : rezervasyon onayı
```

> **Ek bilgi (derste anlatılmadı):** UML sıralama diyagramlarında eşzamanlı çağrı mesajı düz çizgi ve dolu ok ucuyla, dönüş mesajı kesikli çizgi ve açık ok ucuyla çizilir. Bir nesnenin kendine gönderdiği mesaj aynı yaşam çizgisine geri kıvrılan bir okla ve çoğu zaman mevcut etkinlik çubuğunun üstüne binen ikinci bir çubukla gösterilir.

Asistan sıralama diyagramını bitirince istendiği zaman durdurulup soru sorulabileceğini hatırlattı; bu noktada soru gelmedi.

4. Etkinlik Diyagramı

4.1 Etkinlik diyagramı ne işe yarar?

Etkinlik diyagramı, etkinliklerin süreç içindeki dizilişini göstermek ve sistemin akış yönünü belirlemek için kullanılır. Bu örnekte senaryonun akışı çizildi. İki temel öğe kullanıldı:

- **Etkinlik (activity):** Süreçteki her bir adımı gösterir.
- **Karar (decision):** Akışın bir koşula göre ayrıldığı yeri gösterir.

Diyagram bir başlangıç düğümüyle (senaryo start) başlar ve bir bitiş düğümüyle biter. Etkinlikler yukarıdan aşağıya doğru, sistemde istenen adımlar sırasıyla yazılır. Asistan kutuların "dip dibe" olduğunu, ama etkinlik diyagramında adımların alt alta olması gerektiğini belirtti.

4.2 Akış adım adım

- **Yolcu sisteme girer.** Yolcunun seyahat bilgilerini girebilmesi için önce sisteme girmesi gerekir.
- **Yolcu sefer bilgilerini girer.**
- **Sistem sefer bilgisini kontrol eder.** Sistemin, yolcunun girdiği bilgileri denetlemesi gerekir.
- **Karar: sefer var mı?** İstenen tarihte, istenen çıkış yerinden varış yerine bir sefer olmayabilir; örneğin oraya hiç sefer yoktur ya da tarih uymaz. Sistem bunu kontrol eder ve akış ikiye ayrılır:
  - Sefer bilgisi yoksa: "Sefer bilgisi yok, dön."
  - Sefer bilgisi varsa: "Sefer bilgisi göster."
- **Yolcu seferleri görür ve otobüs seçer.**
- **Boş koltuk bilgileri kullanıcıya verilir.** Yolcunun koltuk seçebilmesi için sistemin boş ve dolu koltukları göstermesi gerekir.
- **Kullanıcı koltuğunu seçer.**
- **İletişim bilgileri girilir.** Sistem, koltuğu seçen kullanıcıdan iletişim bilgilerini ister.
- **Rezervasyon yapılır.** Sistem rezervasyonu yapar.
- **Kullanıcı rezervasyonu onaylar.**
- **Bitiş.**

Düz metin olarak:

```
(başla)
  |
Yolcu sisteme girer
  |
Yolcu sefer bilgilerini girer
  |
Sistem sefer bilgisini kontrol eder
  |
<sefer var mı?> --hayır--> Sefer bilgisi yok, dön
  | evet
Sefer bilgisi göster
  |
Yolcu seferleri görür, otobüs seçer
  |
Boş koltuk bilgileri kullanıcıya verilir
```

```
  |
Kullanıcı koltuğunu seçer
  |
İletişim bilgileri girilir
  |
Rezervasyon yapılır
  |
Kullanıcı rezervasyonu onaylar
  |
(bitir)
```

Asistan diyagramı şöyle özetledi: Etkinlik diyagramında çok basitçe, sırayla sistemin ne yaptığı yazılır. Farklı olarak, sistemin karar vermesi gereken yerler vardır. Burada sistem sefer bilgisini kontrol eder; sefer bilgisi yoksa bir yöne, varsa diğer yöne devam eder.

5. Koltuk İçin Durum Diyagramı

5.1 Durum diyagramı ne anlatır?

Son soru, bir koltuğun içinde bulunabileceği durumları ve bunlar arasındaki olası geçişleri ayrıntılandıran bir durum diyagramı istiyordu. Durum diyagramı, bir nesnenin içinde bulunabileceği durumları (state) ve bir durumdan diğerine geçiş kurallarını anlatır. Her durum, tek bir nesnenin belirli bir halini ifade eder. Diyagramda yine bir başlangıç ve bir bitiş düğümü ile durumlar bulunur. Asistan, diyagramın senaryonun belirli bir kesiti, yani rezervasyon işlemi için çizildiğini söyledi.

5.2 Geçişlerin mantığı

Asistan senaryoyu hatırlattı: Yolcu otobüsü seçer ve koltuk bilgilerini görüntüler. Koltuk boşsa yolcu iletişim bilgilerini girer ve koltuk rezerve edilir. Alternatif senaryoya göre, süresinde ödeme yapılmazsa bilet gerçekleşmez ve koltuk yeniden boşa düşer. Ödeme yapılırsa koltuk biletlenir. Yani rezervasyon durumundan iki çıkış vardır.

Geçişler şöyle çizildi:

- Başlangıçtan ilk duruma: “Yolcu otobüsü seçer ve boş koltuk bilgilerini görüntüler.”
- Boş koltuktan rezervasyona: “Yolcu iletişim ve koltuk bilgilerini girer.”
- Rezervasyondan geri boşa: “Yolcu sefer başlangıcına 12 saat kalana kadar ödeme gerçekleştirmedi.”
- Rezervasyondan biletli duruma: “Yolcu ödeme yaptı.”
- Biletli durumdan bitişe: “Koltuk dolu, başkasına verilemez.” Biletlendikten sonra koltuk artık boş görünmez.

Kısa nottaki kareye göre durumların adları Boş, Rezervasyon ve Biletli’dir:

```
(başla) --[yolcu otobüsü seçer ve boş koltuk bilgilerini görüntüler]--> Boş
Boş --[yolcu iletişim ve koltuk bilgilerini girer]--> Rezervasyon
Rezervasyon --[sefer başlangıcına 12 saat kalana kadar ödeme gerçekleştirmedi]--> Boş
Rezervasyon --[yolcu ödeme yaptı]--> Biletli
Biletli --[koltuk dolu, başkasına verilemez]--> (bitir)
```

Kaynak durum	Olay / koşul	Hedef durum
(başlangıç)	Yolcu otobüsü seçer, boş koltukları görüntüler	Boş
Boş	Yolcu iletişim ve koltuk bilgilerini girer	Rezervasyon
Rezervasyon	Sefere 12 saat kalana kadar ödeme yapılmadı	Boş
Rezervasyon	Yolcu ödeme yaptı	Biletli
Biletli	Koltuk dolu, başkasına verilemez	(bitiş)

5.3 12 saat kuralının düzeltilmesi

Asistan geçiş etiketini yazarken önce “yolcu 12 saat içinde” dedi, ardından kendini düzeltti: Kural 12 saat içinde ödeme değil, sefer başlangıcına 12 saat kalana kadar ödemedir. Bu ay-

rım önemlidir. “12 saat içinde” ifadesi süreyi rezervasyon anından başlatır. Senaryodaki kural ise süreyi seferin kalkış saatine göre ölçer: Kalkışa 12 saat kalmış ve ödeme yapılmamışsa rezervasyon düşer.

Asistan durum diyagramını şöyle özetledi: Bu diyagram, yolcunun bilet almasında sistemin bir özeti gibidir. Koltuk boşsa yolcu koltuğu seçer, boş koltuk bilgilerini görüntüler, iletişim ve koltuk bilgilerini girerek rezervasyon yapar. Sefere 12 saat kalana kadar ödemesini yapmazsa rezervasyon yeniden boşa düşer. Ödemesini yaparsa bileti almış olur, koltuk dolar ve başkasına verilmez.

6. Diyagramlar Arası Tutarlılık ve Eksikler

Uygulamanın ana fikri, aynı senaryonun dört diyagramının birbiriyle tutarlı olmasıdır. Sıralama diyagramının sınıf diyagramındaki ilişki ve metotlara uygun çizilmesi açıkça istendi. Durum diyagramındaki geçişler ise senaryonun alternatif akışlarından çıktı. Derste çizilen diyagramlarda bazı eksik ve tutarsızlıklar kaldı.

> **Ders sonrası not:** Sıralama diyagramında rezervasyon ekranından otobüse giden “boş koltukları listele” mesajı kullanıldı ve asistan bunun için “diye bir şeyimiz vardı” dedi. Oysa sınıf diyagramında Otobüs’ün tek metodu `getPlaka()`’dır. Tutarlılık için Otobüs sınıfına bu metot (ör. `bosKoltuklarıListele()`) eklenmelidir. Koltuk’un boşluk durumu için kullanılan `get` metodu da sınıf diyagramında açıkça yazılmalıdır.
>
> **Ders sonrası not:** Sınıf diyagramında `seferler: Sefer()` ve `koltuklar: Koltuk()` yazımı UML’de doğru değildir. Çok değerli nitelik `seferler: Sefer[*]` ya da `koltuklar: List<Koltuk>` gibi gösterilir. Ayrıca `odemeAl()` metodunun önünde görünürlük işareti eksiktir; diğer metotlar gibi + olmalıdır.
>
> **Ders sonrası not:** `seferAra` metodunun dönüş tipi `boolean` yazıldı. Senaryoda bu adımdan sonra seferlerin gösterilmesi beklendiği için `List<Sefer>` gibi bir dönüş tipi daha uygun olur. Rezervasyon sınıfında yolcu, sefer ve koltuk yalnızca `rezervasyonYap` parametresi olarak bulunuyor. Rezervasyonun kime ve hangi koltuğa ait olduğunu saklayabilmesi için bunlar nitelik ya da ilişki olarak tutulmalıdır. Sefer’de hem `otobüs: Otobüs` niteliği hem de Otobüs’e çizilmiş ilişki varsa, ikisinden biri yeterlidir.
>
> **Ders sonrası not:** Sıralama ve etkinlik diyagramlarında ödeme adımı (`odemeYap`, `odemeAl`) ve iki alternatif akış (dolu koltuk seçimi, süresinde ödememe) gösterilmedi. Senaryonun tamamı için etkinlik diyagramına bu kararlar eklenmeli, sıralama diyagramında da alternatifler `alt` çerçevesiyle (combined fragment) gösterilmelidir. Durum diyagramında bu alternatiflerden ikincisi yer alıyor.

Örnek Sorular

Derste öğrenciden soru gelmedi. Aşağıdaki soru, uygulamanın başında okunan ve ders boyunca çözülen alıştırmadır.

Soru (derste sözlü, 00:05:13 – 00:07:59 ve 00:49:59): Bölüm 1’de verilen otobüs bileti senaryosu için (birinci aktör yolcu, ilgili bilet satış sistemi, ön koşul tüm sefer bilgilerinin girilmiş olması, son koşul yolcunun biletini satın almış olması; ana senaryo ve iki alternatif senaryo ile):

- a) Bir UML sınıf şeması çiziniz.
- b) Sınıf şemasındaki ilişki ve metotlara uygun olarak bir UML sıralama diyagramı çiziniz.
- c) Bir etkinlik diyagramı çiziniz.
- d) Bir koltuğun içinde bulunabileceği durumları ve bunlar arasındaki olası geçişleri ayrıntılandıran bir UML durum diyagramı çiziniz.

Çözüm (derste verildi):

- a) 7 sınıf: Yolcu, RezervasyonEkranı, Sefer, Rezervasyon, Ödeme, Otobüs, Koltuk. Nitelikler private, erişim metotları public. Otobüs ile Koltuk arası bileşim, diğerleri ilişki (Bölüm 2).

- b) Nesneler `yolcu`, RezervasyonEkranı, Sefer, `busX`, `koltukY`, Rezervasyon. Mesajlar sınıf metotlarından: `seferAra`, `seferBilgileriniAl`, `gosterSeferBilgileri` (kendine mesaj), `otobüsSec`, `koltukSec`, boş koltuk listeleme, boşluk durumu sorgusu, `rezervasyonYap` ve dönüş mesajları (Bölüm 3).
- c) Başla → sisteme giriş → sefer bilgisi girme → sistem kontrolü → "sefer var mı?" kararı → sefer gösterme → otobüs seçme → boş koltukların verilmesi → koltuk seçme → iletişim bilgileri → rezervasyon → onay → bitir (Bölüm 4).
- d) Boş → Rezervasyon (bilgiler girildi), Rezervasyon → Boş (sefere 12 saat kalana kadar ödeme yok), Rezervasyon → Biletli (ödeme yapıldı), Biletli → bitiş (Bölüm 5).

Dersin Toparlaması

Bu uygulamada tek bir kullanım senaryosundan, internetten otobüs bileti almadan, dört UML diyagramı türetildi. Sınıf diyagramı sistemin statik yapısını verdi: yedi sınıf, nitelikleri, metotları ve aralarındaki ilişkiler; otobüs ile koltuk arasındaki güçlü bağ bileşimle gösterildi. Sıralama diyagramı bu metotları kullanarak nesneler arasındaki mesajlaşmayı zaman sırasıyla gösterdi. Etkinlik diyagramı senaryonun adımlarını ve sistemin karar noktasını akış olarak verdi. Durum diyagramı ise tek bir nesnenin, koltuğun, Boş, Rezervasyon ve Biletli durumları arasındaki geçişlerini ve 12 saat kuralını gösterdi.

Uygulamanın asıl dersi, diyagramların aynı senaryonun farklı görünümleri olduğu ve birbirleriyle tutarlı olmaları gerektiğidir. Sıralama diyagramındaki her mesaj sınıf diyagramında bir metoda, durum diyagramındaki her geçiş senaryodaki bir akışa karşılık gelmelidir.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.